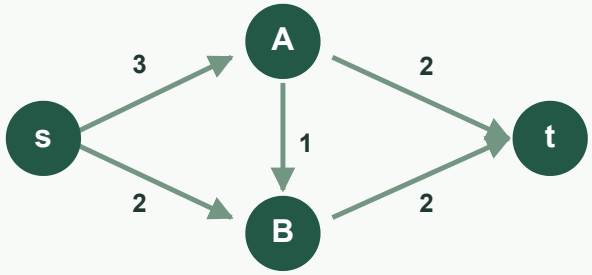
ALGORITHM STUDY NOTE · ASCEND LIVE DEMO

Ford–Fulkerson Algorithm

Find the greatest amount that can move from a source to a destination through a network of capacity-limited connections.

> **Picture a traffic spike.** Requests enter through gateways and travel to workers. Each connection has a maximum rate. A flow network can estimate the most traffic that can be routed at once under those fixed limits.

A small flow network



Edge capacities

s → A: 3
s → B: 2
A → t: 2
B → t: 2
A → B: 1

Send 2 units along **s → A → t**, then 2 along **s → B → t**. Total flow is **4**. It cannot be higher: the two edges entering t have combined capacity 4. This is a maximum flow and a matching minimum cut.

How the algorithm works

1. Start with zero flow.
2. Build the **residual graph**: unused forward capacity is c − f; a reverse edge has capacity f.
3. Find any s-to-t path with positive residual capacity.
4. Push the smallest residual capacity on that path; update forward and reverse edges.
5. Repeat until no augmenting path remains.

Why reverse edges matter

A later path may need to undo part of an earlier choice and reroute that flow. Reverse residual edges make that correction possible.

```
while path exists in residual graph:
    delta = smallest capacity on path
    update forward and reverse edges
    total_flow += delta
```

Try it in the simulator

A SaaS team loses worker capacity during a traffic spike. Change **B → t** from 2 to 1. What is the new maximum flow, and which connection is the bottleneck? Explain your answer before writing code.

Ask the coach: “Use this note to explain residual and reverse edges with this network. Then give me a small request-routing task, let me try first, and critique my reasoning.”

Scope: This is a static capacity model. Real systems also need latency, queues, changing demand, failures, and operational checks. With integer capacities, basic Ford–Fulkerson terminates; using breadth-first search for each path gives the Edmonds–Karp variant.